

Manual de Referencia de Base de Datos

Base de Datos: GestionExtracurricular

Archivo de Origen: proyecto.sql

Vistas Detectadas: 6

Procedimientos Detectados: 17

1. Vistas de Lectura (Views)

Las vistas simplifican las consultas de la aplicación unificando relaciones complejas y calculando métricas dinámicas.

Vista 1: uv_DetalleInscripcionesCursos

Descripción: Muestra el detalle unificado de las inscripciones de estudiantes en los cursos, incluyendo datos del estudiante, nota, estado de inscripción y el instructor.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_DetalleInscripcionesCursos;
-- Filtrar por estudiante:
SELECT * FROM uv_DetalleInscripcionesCursos WHERE Carnet = '20260001';
-- Filtrar por curso:
SELECT * FROM uv_DetalleInscripcionesCursos WHERE IdCurso = 101;
```

Definición SQL:

```
-- Vista para el detalle de inscripciones a cursos
CREATE VIEW uv_DetalleInscripcionesCursos AS
SELECT
    i.IdInscripcion,
    u.Carnet,
    u.NombreCompleto AS NombreEstudiante,
    u.Carrera,
    u.Campus,
    c.IdCurso,
    c.NombreCurso,
    c.FechaInicio,
    c.FechaFin,
    c.Horario,
    c.CupoMaximo,
    i.Nota,
    i.FechaInscripcion,
    i.EstadoInscripcion,
    inst.NombreCompleto AS NombreInstructor
FROM InscripcionCursos i
JOIN Usuarios u ON i.Carnet = u.Carnet
JOIN Cursos c ON i.IdCurso = c.IdCurso
LEFT JOIN Usuarios inst ON c.CarnetInstructor = inst.Carnet;
```

Vista 2: uv_DetalleMembresiasAsociaciones

Descripción: Muestra las solicitudes y membresías vigentes de estudiantes en las distintas asociaciones estudiantiles.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_DetalleMembresiasAsociaciones;
-- Filtrar por asociación:
SELECT * FROM uv_DetalleMembresiasAsociaciones WHERE Acronimo = 'ASEIS';
```

Definición SQL:

```
-- Vista para el detalle de membresías a asociaciones
CREATE VIEW uv_DetalleMembresiasAsociaciones AS
SELECT
    m.IdMembresia,
    u.Carnet,
    u.NombreCompleto AS NombreEstudiante,
    u.Carrera,
    a.IdAsociacion,
    a.Acronimo,
    a.Nombre AS NombreAsociacion,
    m.Rol,
    m.FechaSolicitud,
    m.EstadoValidacion
FROM MembresiaAsociaciones m
JOIN Usuarios u ON m.Carnet = u.Carnet
JOIN Asociaciones a ON m.IdAsociacion = a.IdAsociacion;
```

Vista 3: uv_DetalleInscripcionesDAC

Descripción: Detalla las inscripciones en actividades artísticas y culturales (DAC), relacionando al estudiante, la actividad y su respectivo instructor.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_DetalleInscripcionesDAC;
-- Listar las inscripciones activas:
SELECT * FROM uv_DetalleInscripcionesDAC WHERE EstadoInscripcion = 'Activa';
```

Definición SQL:

```
-- Vista para el detalle de inscripciones DAC
CREATE VIEW uv_DetalleInscripcionesDAC AS
SELECT
    i.IdInscripcionDAC,
    u.Carnet,
    u.NombreCompleto AS NombreEstudiante,
    u.Carrera,
    a.IdActividad,
    a.NombreActividad,
    c.NombreCategoria,
    i.FechaInscripcion,
    i.EstadoInscripcion,
    inst.NombreCompleto AS NombreInstructor
FROM InscripcionesDAC i
JOIN Usuarios u ON i.Carnet = u.Carnet
JOIN ActividadesDAC a ON i.IdActividad = a.IdActividad
JOIN CategoriasDAC c ON a.IdCategoria = c.IdCategoria
LEFT JOIN Usuarios inst ON a.CarnetInstructor = inst.Carnet;
```

Vista 4: uv_DetalleParticipacionDeportiva

Descripción: Muestra la participación de los estudiantes en las selecciones deportivas oficiales y su modalidad.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_DetalleParticipacionDeportiva;
-- Filtrar por disciplina deportiva:
SELECT * FROM uv_DetalleParticipacionDeportiva WHERE Disciplina = 'Fútbol';
```

Definición SQL:

```
-- Vista para el detalle de participación deportiva
CREATE VIEW uv_DetalleParticipacionDeportiva AS
SELECT
    p.IdParticipacion,
    u.Carnet,
    u.NombreCompleto AS NombreEstudiante,
    u.Carrera,
    s.IdSeleccion,
    s.Disciplina,
    s.Rama,
    s.NombreEquipo,
    p.Modalidad,
    p.FechaIngreso
```

```
FROM ParticipacionDeportiva p
JOIN Usuarios u ON p.Carnet = u.Carnet
JOIN SeleccionesDeportivas s ON p.IdSeleccion = s.IdSeleccion;
```

Vista 5: uv_ListarInstructores

Descripción: Lista de catálogo de todos los usuarios registrados que cuentan con el rol de 'Instructor'.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_ListarInstructores;
```

Definición SQL:

```
-- Vista para listar todos los instructores activos en el sistema
CREATE VIEW uv_ListarInstructores AS
SELECT
    u.Carnet,
    u.NombreCompleto AS NombreInstructor,
    u.Carrera,
    u.Campus,
    u.Activo,
    u.CorreoElectronico
FROM Usuarios u
JOIN Roles r ON u.IdRol = r.IdRol
WHERE r.NombreRol = 'Instructor';
```

Vista 6: uv_ListarCursosConCupos

Descripción: Muestra todos los cursos calculando dinámicamente en tiempo real los cupos ocupados (Confirmados) y los cupos disponibles.

Ejemplo de Consulta SQL:

```
SELECT * FROM uv_ListarCursosConCupos;
-- Listar solo cursos con cupo libre disponible:
SELECT * FROM uv_ListarCursosConCupos WHERE CuposDisponibles > 0;
```

Definición SQL:

```
-- Vista para listar los cursos y calcular dinámicamente los cupos ocupados y
disponibles
CREATE VIEW uv_ListarCursosConCupos AS
SELECT
    c.IdCurso,
    c.NombreCurso,
    c.Descripcion,
    c.FechaInicio,
    c.FechaFin,
    c.Horario,
    c.CupoMaximo,
    c.Estado,
    inst.NombreCompleto AS NombreInstructor,
    COUNT(CASE WHEN i.EstadoInscripcion = 'Confirmada' THEN 1 END) AS CuposOcupados,
    c.CupoMaximo - COUNT(CASE WHEN i.EstadoInscripcion = 'Confirmada' THEN 1 END) AS
CuposDisponibles
FROM Cursos c
LEFT JOIN Usuarios inst ON c.CarnetInstructor = inst.Carnet
LEFT JOIN IncripcionCursos i ON c.IdCurso = i.IdCurso
GROUP BY c.IdCurso, c.NombreCurso, c.Descripcion, c.FechaInicio, c.FechaFin, c.
Horario, c.CupoMaximo, c.Estado, inst.NombreCompleto;
```

2. Procedimientos Almacenados (Stored Procedures)

Encapsulan lógica de negocio, manejo de excepciones y control transaccional para garantizar la robustez del sistema.

Procedimiento 1: sp_InscribirEstudianteCurso

Descripción: Realiza la inscripción de un estudiante a un curso. Si es primera vez, inserta el registro. Si ya estaba 'Cancelada', gestiona la re-inscripción. Si el curso está lleno, asigna automáticamente el estado 'En Espera'.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_InscribirEstudianteCurso @Carnet = '20260001', @IdCurso = 101;
```

Definición SQL:

```
-- Procedimiento para inscribir un estudiante a un curso
CREATE PROCEDURE dbo.sp_InscribirEstudianteCurso
    @Carnet VARCHAR(10),
    @IdCurso INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Validar que el estudiante exista
        IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
        BEGIN
            ;THROW 50001, 'El estudiante con el carnet especificado no está
registrado en el sistema.', 1;
        END

        -- Validar que el curso exista
        IF NOT EXISTS (SELECT 1 FROM Cursos WHERE IdCurso = @IdCurso)
        BEGIN
            ;THROW 50002, 'El curso especificado no existe.', 1;
        END

        -- Verificar si ya existe una inscripción previa
        DECLARE @EstadoActual VARCHAR(20) = NULL;
        DECLARE @IdInscripcion INT = NULL;

        SELECT @IdInscripcion = IdInscripcion, @EstadoActual = EstadoInscripcion
        FROM InscripcionCursos
        WHERE Carnet = @Carnet AND IdCurso = @IdCurso;

        IF @EstadoActual IS NOT NULL
        BEGIN
            -- Si ya está confirmado o en espera, no permitir re-inscripción
            IF @EstadoActual IN ('Confirmada', 'En Espera')
            BEGIN
                DECLARE @MsgError VARCHAR(150) = 'El estudiante ya cuenta con una
inscripción activa en este curso. Estado actual: ' + @EstadoActual;
                ;THROW 50003, @MsgError, 1;
            END

            -- Si estaba Cancelada, gestionar la re-inscripción
            IF @EstadoActual = 'Cancelada'
            BEGIN
                DECLARE @CupoMaximo INT;
                DECLARE @InscritosActivos INT;

                SELECT @CupoMaximo = CupoMaximo FROM Cursos WHERE IdCurso = @IdCurso;
                SELECT @InscritosActivos = COUNT(*) FROM InscripcionCursos
                WHERE IdCurso = @IdCurso AND EstadoInscripcion = 'Confirmada';

                DECLARE @NuevoEstado VARCHAR(20) = 'Confirmada';

                -- Si no hay cupo, se manda a lista de espera
                IF @InscritosActivos >= @CupoMaximo
                BEGIN
                    SET @NuevoEstado = 'En Espera';
                END

                UPDATE InscripcionCursos
```

```

        SET EstadoInscripcion = @NuevoEstado,
            FechaInscripcion = GETDATE(),
            Nota = NULL
        WHERE IdInscripcion = @IdInscripcion;
    END
END
ELSE
BEGIN
    -- Realizar una inscripción limpia (el trigger
    TR_InscripcionCursos_Insert se encargará de validar el cupo)
    INSERT INTO InscripcionCursos (Carnet, IdCurso)
    VALUES (@Carnet, @IdCurso);
END

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    ;THROW;
END CATCH
END;

```

Procedimiento 2: sp_CancelarInscripcionCurso

Descripción: Da de baja a un estudiante de un curso cambiando su estado a 'Cancelada'. Esto libera el cupo y promueve automáticamente al estudiante más antiguo en lista de espera.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_CancelarInscripcionCurso @Carnet = '20260001', @IdCurso = 101;
```

Definición SQL:

```

-- Procedimiento para dar de baja a un estudiante de un curso
CREATE PROCEDURE dbo.sp_CancelarInscripcionCurso
    @Carnet VARCHAR(10),
    @IdCurso INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Validar que la inscripción exista y esté activa
        IF NOT EXISTS (
            SELECT 1
            FROM InscripcionCursos
            WHERE Carnet = @Carnet AND IdCurso = @IdCurso AND EstadoInscripcion IN (
'Confirmada', 'En Espera')
        )
        BEGIN
            ;THROW 50004, 'No existe una inscripción activa para el estudiante en el
curso especificado.', 1;
        END

        -- Actualizar el estado a Cancelada (esto disparará
        TR_InscripcionCursos_UpdateDelete para promover al siguiente)
        UPDATE InscripcionCursos
        SET EstadoInscripcion = 'Cancelada'
        WHERE Carnet = @Carnet AND IdCurso = @IdCurso;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 3: sp_RegistrarUsuario

Descripción: Registra un nuevo usuario en la base de datos, validando preventivamente que el nombre del rol asignado sea válido y exista en el catálogo de Roles.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_RegistrarUsuario
    @Carnet = '20260001',
    @NombreCompleto = 'Carlos Pérez',
    @Carrera = 'Ingeniería de Sistemas',
    @Campus = 'Campus Central',
    @CorreoElectronico = 'carlos.perez@universidad.edu',
    @NombreRol = 'Estudiante';
```

Definición SQL:

```
-- Procedimiento para registrar un nuevo usuario con validación de rol
CREATE PROCEDURE dbo.sp_RegistrarUsuario
    @Carnet VARCHAR(10),
    @NombreCompleto VARCHAR(100),
    @Carrera VARCHAR(150),
    @Campus VARCHAR(100),
    @CorreoElectronico VARCHAR(50),
    @NombreRol VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Validar que el rol exista
        DECLARE @IdRol INT = NULL;
        SELECT @IdRol = IdRol FROM Roles WHERE NombreRol = @NombreRol;

        IF @IdRol IS NULL
        BEGIN
            DECLARE @MsgError VARCHAR(150) = 'El rol especificado (' + COALESCE(
@NombreRol, 'NULL') + ') no existe en el sistema.';
            ;THROW 50005, @MsgError, 1;
        END

        -- Validar que el carnet no esté duplicado
        IF EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
        BEGIN
            ;THROW 50006, 'El carnet ingresado ya se encuentra registrado.', 1;
        END

        -- Insertar el nuevo usuario
        INSERT INTO Usuarios (Carnet, NombreCompleto, Carrera, Campus, Activo,
CorreoElectronico, IdRol)
VALUES (@Carnet, @NombreCompleto, @Carrera, @Campus, 1, @CorreoElectronico,
@IdRol);

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;
```

Procedimiento 4: sp_CrearRol

Descripción: Registra un nuevo rol en la tabla de Roles, asegurando que no existan nombres de roles duplicados.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_CrearRol @NombreRol = 'Instructor', @Descripcion = 'Rol para
instructores y docentes';
```

Definición SQL:

```
-- Procedimiento para registrar un nuevo rol en el sistema
```

```

CREATE PROCEDURE dbo.sp_CrearRol
    @NombreRol VARCHAR(50),
    @Descripcion VARCHAR(250)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF EXISTS (SELECT 1 FROM Roles WHERE NombreRol = @NombreRol)
            BEGIN
                ;THROW 50007, 'El rol ya existe en el sistema.', 1;
            END

        INSERT INTO Roles (NombreRol, Descripcion)
        VALUES (@NombreRol, @Descripcion);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 5: sp_ActualizarUsuario

Descripción: Actualiza la información completa de un usuario existente en la base de datos, incluyendo su estado y su rol.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_ActualizarUsuario
    @Carnet = '20260001',
    @NombreCompleto = 'Carlos Pérez',
    @Carrera = 'Ingeniería Industrial',
    @Campus = 'Campus Central',
    @Activo = 1,
    @CorreoElectronico = 'carlos.perez@universidad.edu',
    @NombreRol = 'Estudiante';

```

Definición SQL:

```

-- Procedimiento para actualizar todos los datos de un usuario
CREATE PROCEDURE dbo.sp_ActualizarUsuario
    @Carnet VARCHAR(10),
    @NombreCompleto VARCHAR(100),
    @Carrera VARCHAR(150),
    @Campus VARCHAR(100),
    @Activo BIT,
    @CorreoElectronico VARCHAR(50),
    @NombreRol VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
            BEGIN
                ;THROW 50008, 'El usuario con el carnet especificado no existe.', 1;
            END

        DECLARE @IdRol INT = NULL;
        SELECT @IdRol = IdRol FROM Roles WHERE NombreRol = @NombreRol;
        IF @IdRol IS NULL
            BEGIN
                ;THROW 50009, 'El rol especificado no existe.', 1;
            END

        UPDATE Usuarios
        SET NombreCompleto = @NombreCompleto,
            Carrera = @Carrera,
            Campus = @Campus,
            Activo = @Activo,
            CorreoElectronico = @CorreoElectronico,
            IdRol = @IdRol
        WHERE Carnet = @Carnet;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH

```

```

        IF @@TRANSCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 6: sp_CrearAsociacion

Descripción: Crea una nueva asociación estudiantil en la base de datos.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_CrearAsociacion
    @Acronimo = 'ASEIS',
    @Nombre = 'Asociación de Estudiantes de Ingeniería de Sistemas',
    @AnioFundacion = 2026,
    @Descripcion = 'Grupo de estudiantes para la innovación tecnológica';

```

Definición SQL:

```

-- Procedimiento para crear una nueva asociación
CREATE PROCEDURE dbo.sp_CrearAsociacion
    @Acronimo VARCHAR(20),
    @Nombre VARCHAR(200),
    @AnioFundacion INT,
    @Descripcion NVARCHAR(MAX)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF EXISTS (SELECT 1 FROM Asociaciones WHERE Acronimo = @Acronimo)
            BEGIN
                ;THROW 50010, 'Ya existe una asociación con ese acrónimo.', 1;
            END

        INSERT INTO Asociaciones (Acronimo, Nombre, AnioFundacion, Descripcion)
        VALUES (@Acronimo, @Nombre, @AnioFundacion, @Descripcion);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANSCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 7: sp_CrearCurso

Descripción: Registra un nuevo curso en el sistema. Valida que el instructor asignado sea un usuario activo y posea el rol de 'Instructor'.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_CrearCurso
    @IdCurso = 101,
    @NombreCurso = 'Base de Datos I',
    @Descripcion = 'Curso de bases de datos',
    @FechaInicio = '2026-07-01',
    @FechaFin = '2026-11-30',
    @Horario = 'Lunes y Miércoles 18:00 - 20:00',
    @CupoMaximo = 30,
    @CarnetInstructor = 'INST0001';

```

Definición SQL:

```

-- Procedimiento para registrar un nuevo curso
CREATE PROCEDURE dbo.sp_CrearCurso
    @IdCurso INT,
    @NombreCurso VARCHAR(150),
    @Descripcion NVARCHAR(500),
    @FechaInicio DATE,
    @FechaFin DATE,
    @Horario VARCHAR(100),
    @CupoMaximo INT,

```



```

        @CarnetInstructor VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF EXISTS (SELECT 1 FROM Cursos WHERE IdCurso = @IdCurso)
            BEGIN
                ;THROW 50011, 'El código del curso ya se encuentra registrado.', 1;
            END

        INSERT INTO Cursos (IdCurso, NombreCurso, Descripcion, FechaInicio, FechaFin,
Horario, CupoMaximo, Estado, CarnetInstructor)
VALUES (@IdCurso, @NombreCurso, @Descripcion, @FechaInicio, @FechaFin,
@Horario, @CupoMaximo, 'Activo', @CarnetInstructor);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 8: sp_ActualizarCurso

Descripción: Actualiza los datos de un curso existente, permitiendo cambiar el estado del mismo.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_ActualizarCurso
    @IdCurso = 101,
    @NombreCurso = 'Base de Datos Avanzadas',
    @Descripcion = 'Curso avanzado',
    @FechaInicio = '2026-07-01',
    @FechaFin = '2026-11-30',
    @Horario = 'Lunes y Miércoles 18:00 - 20:00',
    @CupoMaximo = 25,
    @Estado = 'Activo',
    @CarnetInstructor = 'INST0001';

```

Definición SQL:

```

-- Procedimiento para actualizar los datos de un curso
CREATE PROCEDURE dbo.sp_ActualizarCurso
    @IdCurso INT,
    @NombreCurso VARCHAR(150),
    @Descripcion NVARCHAR(500),
    @FechaInicio DATE,
    @FechaFin DATE,
    @Horario VARCHAR(100),
    @CupoMaximo INT,
    @Estado VARCHAR(20),
    @CarnetInstructor VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM Cursos WHERE IdCurso = @IdCurso)
            BEGIN
                ;THROW 50012, 'El curso especificado no existe.', 1;
            END

        UPDATE Cursos
        SET NombreCurso = @NombreCurso,
            Descripcion = @Descripcion,
            FechaInicio = @FechaInicio,
            FechaFin = @FechaFin,
            Horario = @Horario,
            CupoMaximo = @CupoMaximo,
            Estado = @Estado,
            CarnetInstructor = @CarnetInstructor
        WHERE IdCurso = @IdCurso;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    END CATCH
END;

```

```

        ;THROW;
    END CATCH
END;

```

Procedimiento 9: sp_CrearSeleccionDeportiva

Descripción: Crea un equipo o selección deportiva en una disciplina y rama determinada.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_CrearSeleccionDeportiva @Disciplina = 'Baloncesto', @Rama =
'Femenina', @NombreEquipo = 'Tigresas FC';

```

Definición SQL:

```

-- Procedimiento para crear una nueva selección deportiva
CREATE PROCEDURE dbo.sp_CrearSeleccionDeportiva
    @Disciplina VARCHAR(50),
    @Rama VARCHAR(20),
    @NombreEquipo VARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF EXISTS (SELECT 1 FROM SeleccionesDeportivas WHERE NombreEquipo =
@NombreEquipo)
            BEGIN
                ;THROW 50013, 'El nombre de equipo deportivo ya se encuentra
registrado.', 1;
            END

        INSERT INTO SeleccionesDeportivas (Disciplina, Rama, NombreEquipo)
        VALUES (@Disciplina, @Rama, @NombreEquipo);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 10: sp_CrearCategoriaDAC

Descripción: Crea una nueva categoría para agrupar talleres artísticos y de cultura (DAC).

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_CrearCategoriaDAC @NombreCategoria = 'Música', @Descripcion =
'Talleres de canto e instrumentos';

```

Definición SQL:

```

-- Procedimiento para crear una nueva categoría DAC
CREATE PROCEDURE dbo.sp_CrearCategoriaDAC
    @NombreCategoria VARCHAR(100),
    @Descripcion NVARCHAR(500)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF EXISTS (SELECT 1 FROM CategoriasDAC WHERE NombreCategoria =
@NombreCategoria)
            BEGIN
                ;THROW 50014, 'Ya existe una categoría artística con ese nombre.', 1;
            END

        INSERT INTO CategoriasDAC (NombreCategoria, Descripcion)
        VALUES (@NombreCategoria, @Descripcion);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    END CATCH
END;

```

```

        ;THROW;
    END CATCH
END;

```

Procedimiento 11: sp_CrearActividadDAC

Descripción: Crea una actividad artística DAC (ej. Teatro, Canto) asignándole un instructor válido y activo en el sistema.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_CrearActividadDAC
    @IdCategoria = 1,
    @NombreActividad = 'Taller de Guitarra',
    @Descripcion = 'Clases prácticas de guitarra acústica',
    @CarnetInstructor = 'INST0002';

```

Definición SQL:

```

-- Procedimiento para crear una nueva actividad DAC
CREATE PROCEDURE dbo.sp_CrearActividadDAC
    @IdCategoria INT,
    @NombreActividad VARCHAR(150),
    @Descripcion NVARCHAR(MAX),
    @CarnetInstructor VARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM CategoriasDAC WHERE IdCategoria = @IdCategoria)
            BEGIN
                ;THROW 50015, 'La categoría DAC especificada no existe.', 1;
            END

        INSERT INTO ActividadesDAC (IdCategoria, NombreActividad, Descripcion,
Estado, CarnetInstructor)
VALUES (@IdCategoria, @NombreActividad, @Descripcion, 'Activo',
@CarnetInstructor);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 12: sp_InscribirMembresiaAsociacion

Descripción: Registra una solicitud de membresía de un estudiante para una asociación. Si la solicitud anterior fue rechazada, reactiva la membresía colocándola en estado 'Pendiente'.

Ejemplo de Uso (Ejecución):

```

EXEC dbo.sp_InscribirMembresiaAsociacion @Carnet = '20260001', @IdAsociacion =
1, @Rol = 'Miembro';

```

Definición SQL:

```

-- Procedimiento para afiliar a un estudiante a una asociación
CREATE PROCEDURE dbo.sp_InscribirMembresiaAsociacion
    @Carnet VARCHAR(10),
    @IdAsociacion INT,
    @Rol VARCHAR(50) = 'Miembro'
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
            BEGIN
                ;THROW 50016, 'El carnet ingresado no existe en el sistema.', 1;
            END
        IF NOT EXISTS (SELECT 1 FROM Asociaciones WHERE IdAsociacion = @IdAsociacion)

```

```

        BEGIN
        ;THROW 50017, 'La asociación especificada no existe.', 1;
        END

        DECLARE @EstadoActual VARCHAR(20) = NULL;
        DECLARE @IdMembresia INT = NULL;
        SELECT @IdMembresia = IdMembresia, @EstadoActual = EstadoValidacion
        FROM MembresiaAsociaciones WHERE Carnet = @Carnet AND IdAsociacion =
@IdAsociacion;

        IF @EstadoActual IS NOT NULL
        BEGIN
            IF @EstadoActual IN ('Pendiente', 'Aprobada')
            BEGIN
                ;THROW 50018, 'El estudiante ya cuenta con una solicitud activa
o membresía en esta asociación.', 1;
            END
            IF @EstadoActual = 'Rechazada'
            BEGIN
                UPDATE MembresiaAsociaciones
                SET EstadoValidacion = 'Pendiente', Rol = COALESCE(@Rol, 'Miembro'),
FechaSolicitud = GETDATE()
                WHERE IdMembresia = @IdMembresia;
            END
        END
        ELSE
        BEGIN
            INSERT INTO MembresiaAsociaciones (Carnet, IdAsociacion, Rol,
EstadoValidacion)
            VALUES (@Carnet, @IdAsociacion, COALESCE(@Rol, 'Miembro'), 'Pendiente');
        END
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 13: sp_CancelarMembresiaAsociacion

Descripción: Rechaza o da de baja la membresía de un estudiante en una asociación, cambiando su estado a 'Rechazada'.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_CancelarMembresiaAsociacion @Carnet = '20260001', @IdAsociacion = 1;
```

Definición SQL:

```

-- Procedimiento para rechazar o dar de baja la membresía de una asociación
CREATE PROCEDURE dbo.sp_CancelarMembresiaAsociacion
    @Carnet VARCHAR(10),
    @IdAsociacion INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM MembresiaAsociaciones WHERE Carnet = @Carnet
AND IdAsociacion = @IdAsociacion AND EstadoValidacion IN ('Pendiente', 'Aprobada'))
        BEGIN
            ;THROW 50019, 'No existe una membresía activa o pendiente para este
estudiante en la asociación.', 1;
        END

        UPDATE MembresiaAsociaciones
        SET EstadoValidacion = 'Rechazada'
        WHERE Carnet = @Carnet AND IdAsociacion = @IdAsociacion;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 14: sp_InscribirParticipacionDeportiva

Descripción: Registra la participación de un estudiante en un equipo deportivo en una modalidad de competencia.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_InscribirParticipacionDeportiva @Carnet = '20260001', @IdSeleccion
= 1, @Modalidad = 'Torneos Nacionales';
```

Definición SQL:

```
-- Procedimiento para registrar participación deportiva
CREATE PROCEDURE dbo.sp_InscribirParticipacionDeportiva
    @Carnet VARCHAR(10),
    @IdSeleccion INT,
    @Modalidad VARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
            BEGIN
                ;THROW 50020, 'El carnet del estudiante no existe.', 1;
            END
        IF NOT EXISTS (SELECT 1 FROM SeleccionesDeportivas WHERE IdSeleccion =
@IdSeleccion)
            BEGIN
                ;THROW 50021, 'La selección deportiva especificada no existe.', 1;
            END
        IF EXISTS (SELECT 1 FROM ParticipacionDeportiva WHERE Carnet = @Carnet AND
IdSeleccion = @IdSeleccion)
            BEGIN
                ;THROW 50022, 'El estudiante ya participa en esta selección
deportiva.', 1;
            END

        INSERT INTO ParticipacionDeportiva (Carnet, IdSeleccion, Modalidad)
VALUES (@Carnet, @IdSeleccion, @Modalidad);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;
```

Procedimiento 15: sp_CancelarParticipacionDeportiva

Descripción: Retira de forma física (elimina) el registro de participación de un estudiante en una selección deportiva.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_CancelarParticipacionDeportiva @Carnet = '20260001', @IdSeleccion =
1;
```

Definición SQL:

```
-- Procedimiento para retirar participación deportiva (eliminación)
CREATE PROCEDURE dbo.sp_CancelarParticipacionDeportiva
    @Carnet VARCHAR(10),
    @IdSeleccion INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM ParticipacionDeportiva WHERE Carnet = @Carnet
AND IdSeleccion = @IdSeleccion)
            BEGIN
                ;THROW 50023, 'El estudiante no tiene registrada participación en
esta selección deportiva.', 1;
            END

        DELETE FROM ParticipacionDeportiva
WHERE Carnet = @Carnet AND IdSeleccion = @IdSeleccion;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;
```

```

END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    ;THROW;
END CATCH
END;

```

Procedimiento 16: sp_InscribirActividadDAC

Descripción: Inscribe a un estudiante en un taller DAC. Si se había retirado de la actividad previamente, la reactiva a estado 'Activa'.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_InscribirActividadDAC @Carnet = '20260001', @IdActividad = 1;
```

Definición SQL:

```

-- Procedimiento para inscribir en actividad DAC
CREATE PROCEDURE dbo.sp_InscribirActividadDAC
    @Carnet VARCHAR(10),
    @IdActividad INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM Usuarios WHERE Carnet = @Carnet)
            BEGIN
                ;THROW 50024, 'El carnet del estudiante no existe.', 1;
            END
        IF NOT EXISTS (SELECT 1 FROM ActividadesDAC WHERE IdActividad = @IdActividad)
            BEGIN
                ;THROW 50025, 'La actividad artistica (DAC) especificada no existe.',
1;
            END

        DECLARE @EstadoActual VARCHAR(20) = NULL;
        DECLARE @IdInscripcionDAC INT = NULL;
        SELECT @IdInscripcionDAC = IdInscripcionDAC, @EstadoActual =
EstadoInscripcion
        FROM InscripcionesDAC WHERE Carnet = @Carnet AND IdActividad = @IdActividad;

        IF @EstadoActual IS NOT NULL
            BEGIN
                IF @EstadoActual IN ('Activa', 'Finalizada')
                    BEGIN
                        ;THROW 50026, 'El estudiante ya cuenta con una inscripción
activa o finalizada para esta actividad.', 1;
                    END
                IF @EstadoActual = 'Retirada'
                    BEGIN
                        UPDATE InscripcionesDAC
                        SET EstadoInscripcion = 'Activa', FechaInscripcion = GETDATE()
                        WHERE IdInscripcionDAC = @IdInscripcionDAC;
                    END
            END
        ELSE
            BEGIN
                INSERT INTO InscripcionesDAC (Carnet, IdActividad)
                VALUES (@Carnet, @IdActividad);
            END
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH
END;

```

Procedimiento 17: sp_CancelarInscripcionDAC

Descripción: Retira a un estudiante de una actividad DAC, cambiando su estado de inscripción a 'Retirada'.

Ejemplo de Uso (Ejecución):

```
EXEC dbo.sp_CancelarInscripcionDAC @Carnet = '20260001', @IdActividad = 1;
```

Definición SQL:

```
-- Procedimiento para cancelar inscripción DAC (Retirado)
CREATE PROCEDURE dbo.sp_CancelarInscripcionDAC
    @Carnet VARCHAR(10),
    @IdActividad INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        IF NOT EXISTS (SELECT 1 FROM InscripcionesDAC WHERE Carnet = @Carnet AND
IdActividad = @IdActividad AND EstadoInscripcion = 'Activa')
            BEGIN
                ;THROW 50027, 'No existe una inscripción activa para este estudiante
en la actividad artística.', 1;
            END

            UPDATE InscripcionesDAC
            SET EstadoInscripcion = 'Retirada'
            WHERE Carnet = @Carnet AND IdActividad = @IdActividad;
            COMMIT TRANSACTION;
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
            ;THROW;
        END CATCH
    END;
```